

RCAEval: A Benchmark for Root Cause Analysis of Microservice Systems with Telemetry Data

Luan Pham
luan.pham@rmit.edu.au
RMIT University
Melbourne, Australia
University of New South Wales
Sydney, Australia

Hongyu Zhang
hongyu.zhang@newcastle.edu.au
The University of Newcastle
Newcastle, Australia

Huong Ha
huong.ha@rmit.edu.au
RMIT University
Melbourne, Australia

Flora Salim
flora.salim@unsw.edu.au
University of New South Wales
Sydney, Australia

Xiuzhen Zhang
xiuzhen.zhang@rmit.edu.au
RMIT University
Melbourne, Australia

Abstract

Root cause analysis (RCA) for microservice systems has gained significant attention in recent years. However, there is still no standard benchmark that includes large-scale datasets and supports comprehensive evaluation environments. In this paper, we introduce RCAEval, an open-source benchmark that provides datasets and an evaluation environment for RCA in microservice systems. First, we introduce three comprehensive datasets comprising 735 failure cases collected from three microservice systems, covering various fault types observed in real-world failures. Second, we present a comprehensive evaluation framework that includes fifteen reproducible baselines covering a wide range of RCA approaches, with the ability to evaluate both coarse-grained and fine-grained RCA. We hope that this ready-to-use benchmark will enable researchers and practitioners to conduct extensive analysis and pave the way for robust new solutions for RCA of microservice systems.

CCS Concepts

• Software and its engineering → Software reliability.

Keywords

Benchmark, Root Cause Analysis, Microservices, Telemetry Data

ACM Reference Format:

Luan Pham, Hongyu Zhang, Huong Ha, Flora Salim, and Xiuzhen Zhang. 2025. RCAEval: A Benchmark for Root Cause Analysis of Microservice Systems with Telemetry Data. In *Companion Proceedings of the ACM Web Conference 2025 (WWW Companion '25)*, April 28-May 2, 2025, Sydney, NSW, Australia. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3701716.3715290>

1 Introduction

Root cause analysis (RCA) for microservice systems is an important problem that has been studied recently, as failures are inevitable,

and ensuring the reliability of microservice systems is critical. RCA aims to analyse the available telemetry data (i.e., metrics, logs, and traces) of the system during failure periods to identify the root cause service and root cause indicators (e.g., specific metrics or logs pointing to the root cause). This field has gained significant attention recently [6, 9, 15, 16, 18]. However, there is still no standard benchmark that includes large-scale datasets and a comprehensive evaluation framework [4]. This limitation leads to inconsistent evaluations in RCA studies, hindering understanding and impeding progress in the field [4]. For example, existing studies [8, 10, 13] typically evaluate their methods on only 1-2 systems with 2-3 fault types. Eadro [9] uses a dataset with an unrealistic load (2-3 requests per second). Existing open-source RCA resources also suffer from several limitations. For example, PyRCA [14] offers only a limited set of metric-based RCA methods and relies on synthetic datasets. The AIOps 2020 dataset [12] contains failures with metrics and traces but omits valuable log information. Pham et al. [6] evaluates only on metric-based RCA methods. As a result, existing resources are often inadequate for benchmarking purposes, hampering the development of new RCA approaches, see Table 1.

Table 1: Comparison of studies.

Study	Fault Types	Metric	Log	Trace
PyRCA [14]	Synthetic	✓	-	-
AIOps 2020 [12]	Resource, Network	✓	-	✓
Pham et al. [6]	Resource, Network	✓	-	-
RCAEval (ours)	Resource, Network, Code-level	✓	✓	✓

In this work, we introduce RCAEval, a benchmark including three datasets and a comprehensive evaluation environment. First, our datasets include 735 failure cases collected from three systems, covering 11 fault types observed in real-world failures. We collected multi-source telemetry data (i.e., metrics, logs, and traces), supporting a variety of RCA approaches (e.g., metric-based, multi-source RCA). Second, we open-source our evaluation framework, which includes several reproducible baselines. The prior version of this framework was used to evaluate metric-based RCA [6]. In this work, we have upgraded it to support trace-based and multi-source RCA. We also provide preliminary experiments, highlighting the need for further investigation on a robust RCA approach. We have open-sourced our benchmark, including the datasets and the evaluation framework at <https://github.com/phamquiluan/RCAEval>.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW Companion '25*, Sydney, NSW, Australia
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1331-6/25/04
<https://doi.org/10.1145/3701716.3715290>

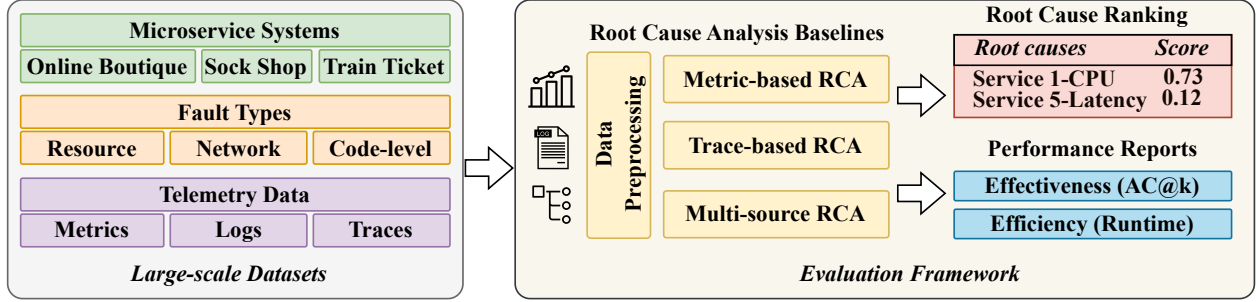


Figure 1: Overview of the RCAEval benchmark.

2 Background & Related Work

In microservice systems, *failures* refer to a service’s inability to perform its intended functions, while *faults* represent the underlying causes of such failures (e.g., memory leaks). *Root cause analysis* (RCA) seeks to pinpoint the root causes of failures by analyzing multi-source telemetry data (i.e., *metrics*, *logs*, and *traces*) [6, 15, 16].

A major limitation in this field is the absence of a reproducible and open-source public benchmark for evaluating RCA in practical scenarios. Most RCA studies evaluate their methods using limited faults on limited systems [4, 6]. For example, some works [8, 9, 18] inject 2-3 faults into 1-2 systems, resulting in limited datasets. Others assess their solutions using private data, such as AWS [8] or Oracle [10]. This lack of transparent and reproducible resources hinders progress and prevents fair evaluation of new RCA approaches.

There have been some related works that introduce datasets or evaluation frameworks, but all of them suffer from several limitations, see Table 1. PyRCA [14] from Salesforce supports only metric-based RCA and relies on synthetic datasets. Our prior work [6] demonstrates that performance on synthetic datasets often fails to reflect RCA performance on real systems. Li et al. [12] introduced datasets with metrics and traces on private systems but omitting logs and did not provide a benchmarking framework. To address these limitations, in this study, we provide a benchmark consisting of three RCA datasets and an open-source evaluation environment.

3 Datasets

RCAEval benchmark includes **three datasets**: RE1, RE2, and RE3, designed to comprehensively support benchmarking RCA in microservice systems. Together, our three datasets feature 735 failure cases collected from three microservice systems (described in Section 3.1) and including 11 fault types (described in Section 3.2). Each failure case also includes annotated root cause service and root cause indicator (e.g., specific metric or log indicating the root cause). The statistics of the datasets are presented in Table 2.

RE1 Dataset. The RE1 dataset, introduced in our prior work on metric-based RCA [6], contains 375 failure cases collected from three microservice systems (125 cases per system). These cases combine five fault types across five services, and five repetitions per fault-service pair. The RE1 dataset exclusively contains metrics data, supporting the development of metric-based RCA methods. The fault types in RE1 include CPU, MEM, DISK, DELAY, LOSS (see Section 3.2). The number of metrics ranges from 49 to 212, depending on the system size, with smaller systems (e.g., Online Boutique, Sock Shop) having fewer metrics compared to larger system (Train Ticket). This dataset does not include logs or traces.

RE2 Dataset. The RE2 dataset, newly collected for this study, supports the development of multi-source RCA methods. It includes 270 failure cases (90 cases per system), combining six fault types across five services, and three repetitions per fault-service pair. RE2 provides multi-source telemetry data, including metrics, logs, and traces. The number of metrics ranges from 77 to 327 per failure case. Each system generates a substantial volume of logs from (8.6 to 26.9 million lines), and traces (39.6 to 76.7 million traces). The fault types include those in RE1 and an additional SOCKET fault.

RE3 Dataset. The RE3 dataset, also newly collected, focuses on supporting multi-source RCA methods with the ability to diagnose code-level faults. It has 90 failure cases (30 per system), involving code-level faults. The fault types in RE3 are F1, F2, F3, F4, F5 (see Section 3.2). Like RE2, RE3 includes multi-source telemetry data (metrics, logs, and traces). This dataset emphasizes diagnosing code-level faults through telemetry data, e.g., leveraging stack traces in logs or response code in traces to pinpoint root causes, making it invaluable for advancing multi-source RCA methods.

Table 2: Statistics of the RCAEval datasets.

Dataset	Systems	Fault Types	Cases	Metrics	Logs (millions)	Traces (millions)
RE1 [6]	3	3 Resource, 2 Network	375	49–212	N/A	N/A
RE2	3	4 Resource, 2 Network	270	77–376	8.6–26.9	39.6–76.7
RE3	3	5 Code-level	90	68–322	1.7–2.7	4.5–4.7

3.1 Microservice Systems

We collect our three datasets from three microservice systems, ranging from 12 to 64 services. These systems are used in our previous works for evaluating RCA methods [6, 16].

1) Online Boutique. The Online Boutique system [2], developed by Google, consists of 12 services forming an e-commerce application where users can browse, add, and purchase items. The services communicate with each other using the gRPC protocol.

2) Sock Shop. The Sock Shop system [3], developed by Weaveworks, is a sock-selling e-commerce application comprising 15 services that communicate via HTTP requests.

3) Train Ticket. Train Ticket [1] is a ticket booking system with 64 services, featuring both synchronous and asynchronous communication. Compared to Sock Shop and Online Boutique, Train Ticket has more complex call chains. To the best of our knowledge, it is the largest benchmark microservice systems.

While no single system can fully capture the diversity of real-world environments, the developers of these systems have intentionally included diverse features, such as multiple programming languages (e.g., Java, Go, Python, C#) and communication protocols (e.g., HTTP, gRPC), to emulate real-world complexity.

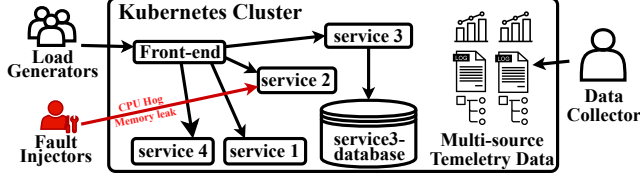


Figure 2: Illustration of our data collection setup.

3.2 Fault Types

Our three datasets consist of 11 fault types (4 resource faults, 2 network faults, and 5 code-level faults). In this section, we describe these faults and the way we introduce them into the microservice systems. The RE1 dataset, which includes 3 resource faults and 2 network faults, was used in our previous metric-based RCA works[6]. In this study, we introduce two additional datasets, RE2 and RE3, which include one additional resource fault and 5 new code-level faults, covering a broader range of faults commonly found in open-source projects [5]. To the best of our knowledge, our datasets are the first to cover code-level faults for RCA in microservice systems.

1) Resource Faults. We introduce four resource faults into the running container (i.e., service instance) using stress-ng: CPU hog (**CPU**), Memory leak (**MEM**), Disk stress (**DISK**), and Socket stress (**SOCK**). Symptoms of resource faults may include observable changes in resource usage of co-located containers, increased latency, and time-out requests. The system may crash when resources are severely constrained. The root cause indicator for these faults is the metric specifying resource usage (e.g., for a CPU hog, the root cause indicator is the container’s CPU usage metric).

2) Network Faults. We use traffic control (tc) to intercept the network packets of the running container, introducing delay variations (**DELAY**) or randomly dropping packets (**DROP**). Symptoms of network faults may include increased latency metrics and error response codes in traces/metrics of the affected service. The root cause indicator for a DELAY fault is the latency metric, while for a LOSS fault, it is the metric showing failed requests and/or error response codes in the traces of the corresponding container.

3) Code-Level Faults. We modify the source code of random services to introduce five bugs commonly found in open-source projects [5]: Incorrect parameter values (**F1**), Missing parameters (**F2**), Missing Function Call (**F3**), Incorrect Return Values (**F4**), and Missing Exception Handlers (**F5**). Symptoms of code-level faults may include increased failed requests, error response codes in traces, higher latency, and stack traces emitted in logs. The root cause of code-level faults is determined using the stack traces in logs of the corresponding service, which indicate the faulty line of code. If stack traces are unavailable, the root cause indicator may be derived from error logs or response codes of the affected service.

3.3 Telemetry Data Collection Process

We deploy three microservice systems to Kubernetes clusters and generate a random load of 10–200 requests per second across all services. We use standard, well-known open-source tools to monitor and collect telemetry data. To gather metrics, we use Prometheus, cAdvisor, and Istio to monitor and collect both application-level and resource-level metrics. For logs, we use Vector and Loki to gather logs from all service instances and store them in Elasticsearch. Traces are collected using Jaeger and sent to Elasticsearch

Table 3: Metrics.

time	cart_cpu	cart_mem
17336	0.216	0.352
17337	0.115	0.401
17338	0.116	0.386
17339	0.118	0.398

Table 4: Logs.

time	service	message
17336	cart	GetCart called...
17337	currency	Getting values...
17338	frontend	request complete.
17339	frontend	request started.

Table 5: Traces.

time	id	service	operation	duration
17336	c8fb...	frontend	GetCurrencies	497
17337	60ef...	currency	Convert	102
17338	4a93...	frontend	GetProduct	1310
17338	fe23...	product	ListProducts	56

for storage, see Fig. 2. We allow the microservice systems to run normally to collect normal telemetry data. Then, we inject a fault into a randomly selected running service and collect the abnormal telemetry data. To ensure data quality, we engaged a DevOps engineer with five years of experience in microservices to assist with system deployment, data collection, and data verification.

3.4 Data Format

The raw telemetry data collected is stored as CSV files. The key structures are presented in Tables 3, 4, and 5. A complete dataset with full structure can be downloaded from our GitHub repository. Metrics are stored as time series data, with each row corresponding to a timestamp at which the metrics were collected. Logs for a failure case are stored in a single CSV file, with each row containing the timestamp, the service name, and the corresponding log message. Similarly, traces for a failure case are stored in a CSV file, where each row includes the timestamp, trace_id, span_id, service name, operation, duration, and response code if available.

4 Evaluation Framework

To ensure the comprehensiveness of RCAEval, we also provide an evaluation framework as an open-source library alongside our datasets. Our evaluation framework includes fifteen baselines covering a wide range of state-of-the-art RCA approaches and offers functionalities for data processing and benchmark evaluation at both coarse-grained and fine-grained levels. The RCAEval evaluation framework is an extension of our previous work [6], which focused on metric-based RCA and coarse-grained RCA. In this work, we expand it by incorporating trace-based and multi-source RCA baselines. RCAEval is released as an open-source library and can be installed via PyPI. Comprehensive documentation on installing and using the framework with our datasets, as well as guidance on extending it with new methods and datasets, is available on our GitHub repository. The documentation also includes basic usage examples and detailed instructions for ensuring reproducibility.

4.1 Evaluation Baselines

Our evaluation framework features 15 baselines covering a variety of state-of-the-art RCA methods. **Metric-based RCA baselines** include causal inference-based methods such as RUN, CausalRCA, CIRCA, RCD, MicroCause, EasyRCA, MSCRED, as well as non-causal methods such as BARO, and ϵ -Diagnosis [6, 8, 10, 14]. **Trace-based RCA baselines** include TraceRCA and MicroRank [11, 17]. **Multi-source RCA baselines** include PDiagnose, multi-source BARO, multi-source RCD, multi-source CIRCA [8, 10, 16]. For baselines like RUN, CausalRCA, CIRCA, RCD, MicroCause, EasyRCA, MSCRED, BARO, ϵ -Diagnosis, MicroRank, and TraceRCA, we adapt their available implementations and use the default hyperparameter settings recommended in their respective papers. We verified their correctness by reproducing the results presented in the original and related studies. For multi-source BARO, multi-source RCD, and multi-source CIRCA, we updated their source code to handle time series data from logs and traces. For PDiagnose, we follow previous

Table 6: RCA performance of eight baselines on the Train Ticket system of the RE2 dataset, across six fault types.

Data Source	Method	CPU			MEM			DISK			SOCKET			DELAY			LOSS			AVERAGE		
		AC@1	AC@3	Avg@5	AC@1	AC@3	Avg@5	AC@1	AC@3	Avg@5	AC@1	AC@3	Avg@5	AC@1	AC@3	Avg@5	AC@1	AC@3	Avg@5	AC@1	AC@3	Avg@5
Metric	BARO	0.47	0.8	0.72	0.93	1	0.99	1	1	1	0.6	0.87	0.83	0.47	0.67	0.63	0.53	0.6	0.64	0.67	0.82	0.8
	CausalRCA	0.4	0.63	0.59	0.1	0.27	0.24	0.43	0.83	0.75	0.23	0.5	0.45	0.13	0.23	0.21	0.03	0.37	0.33	0.22	0.47	0.43
	CIRCA	0.27	0.27	0.28	0.47	0.73	0.68	0.53	0.67	0.64	0.27	0.53	0.52	0.2	0.27	0.28	0.2	0.33	0.35	0.32	0.47	0.46
	MicroCause	0.19	0.44	0.4	0	0.09	0.07	0.4	0.4	0.4	0	0.17	0.15	0	0.22	0.13	0	0	0.07	0.1	0.22	0.2
	RCD	0.13	0.13	0.16	0.07	0.07	0.07	0	0.07	0.05	0.13	0.33	0.29	0.13	0.13	0.15	0.07	0.07	0.07	0.09	0.13	0.13
Trace	MicroRank	0.21	0.43	0.34	0.25	0.38	0.33	0	0.36	0.27	0.3	0.4	0.36	0.08	0.31	0.23	0.14	0.36	0.3	0.16	0.37	0.31
	TraceRCA	0.64	0.79	0.74	0.63	0.88	0.83	0.64	0.71	0.74	0.6	0.8	0.76	0.85	0.85	0.88	0.57	0.71	0.67	0.66	0.79	0.77
Multi-Source	BARO	0.47	0.8	0.75	0.93	1	0.99	1	1	1	0.6	0.8	0.79	0.47	0.67	0.61	0.67	0.67	0.71	0.69	0.82	0.81
	CIRCA	0	0.07	0.09	0.07	0.13	0.21	0	0.07	0.09	0.07	0.13	0.16	0.07	0.07	0.07	0.13	0.2	0.17	0.06	0.11	0.13
	PDiagnose	0.6	0.87	0.81	0.4	0.47	0.48	0.33	0.73	0.69	0.33	0.67	0.6	0.87	0.87	0.87	0.33	0.6	0.57	0.48	0.7	0.67
	RCD	0.17	0.84	0.72	0	0.44	0.39	0.07	0.76	0.62	0.21	0.77	0.66	0.05	0.28	0.25	0.07	0.75	0.6	0.1	0.64	0.54

works [7, 18, 19] to implement it since its source code is unavailable. Previous works such as PyRCA [14] and Pham et al. [15] offer only a limited set of metric-based RCA methods, while our framework provides a more comprehensive set of baselines by also including trace-based and multi-source RCA methods.

4.2 Evaluation Metrics

We support evaluation at both the coarse-grained level (i.e., root cause service) and the fine-grained level (i.e., root cause indicator). The evaluation script executes the analysis and stores the results in a report file. We currently support two standard metrics [6, 16]: $AC@k$ and $Avg@k$ to measure the RCA performance. Given a set of failure cases A , $AC@k$ is calculated as $AC@k = \frac{1}{|A|} \sum_{a \in A} \frac{\sum_{i \leq k} R^a[i] \in V_{rc}^a}{\min(k, |V_{rc}^a|)}$, where $R^a[i]$ is the i th ranking result for the failure case a by an RCA method, and V_{rc}^a is the true root cause set of case a . $AC@k$ represents the probability the top k results of the given method include the true root causes. Its values range from 0 to 1, with higher values indicating better performance. $Avg@k$, which shows the overall RCA performance, is measured as $Avg@k = \frac{1}{k} \sum_{j=1}^k AC@j$.

5 Preliminary Experiments

We conduct preliminary experiments on our benchmark to evaluate the performance of existing baselines on the collected datasets, highlighting both the potential and challenges in the field. Due to space constraints, we select 11 baselines: 5 metric-based RCA methods (BARO, CausalRCA, CIRCA, MicroCause), 2 trace-based RCA methods (MicroRank, TraceRCA), and 4 multi-source RCA methods (PDiagnose, multi-source CIRCA, multi-source RCD, multi-source BARO). These methods are used to diagnose 4 resource faults (CPU, MEM, DISK, SOCK) and 2 network faults (DELAY, LOSS) using data collected from the Train Ticket system in the RE2 dataset. The RCA performance is evaluated using the $AC@1$, $AC@3$, and $Avg@5$ metrics, with coarse-grained results presented in Table 6. A demonstration of diagnosing root causes for code-level faults (e.g., F1 to F5) is available on our GitHub repository.

Our preliminary results show that there is still ample room for further improvement. Existing methods mostly obtain moderate results. For example, CIRCA and RCD obtain the best average $Avg@5$ score of 0.46 and 0.54, respectively. Notably, BARO shows encouraging results when obtaining high accuracy in diagnosing the resource fault (e.g. DISK), however, it shows limitations when dealing with network faults (e.g. DELAY, LOSS). Hence, we believe further research is needed to develop a holistic RCA solution.

6 Conclusion

This paper presents RCAEval, which includes three datasets collected from three microservice systems covering 735 failure cases and 11 fault types, along with a comprehensive evaluation framework with a wide range of reproducible RCA baselines. To the best of our knowledge, this is the first comprehensive benchmark for RCA of microservices. We hope that this benchmark will be useful for industry practitioners and academic researchers in the field.

Acknowledgments

This research was supported by the Australian Research Council Discovery Project (DP220103044). We would like to thank RMIT Race Hub for providing the computing support via the RMAS scheme. We also extend our gratitude to Dr. Kien Do for his generous support through AWS Cloud Credit.

References

- [1] 2024. <https://github.com/FudanSELab/train-ticket>.
- [2] 2024. <https://github.com/GoogleCloudPlatform/microservices-demo>.
- [3] 2024. <https://github.com/microservices-demo/microservices-demo>.
- [4] Qian Cheng et al. 2023. Ai for it operations (aiops) on cloud platforms: Reviews, opportunities and challenges. *arXiv:2304.04661* (2023).
- [5] Domenico Cotroneo et al. 2019. How bad can a bug get? an empirical analysis of software failures in the openstack cloud computing platform. In *FSE'19*.
- [6] Pham et al. 2024. Root Cause Analysis for Microservice System based on Causal Inference: How Far Are We?. In *IEEE/ACM ASE'24*.
- [7] Chuanjia Hou et al. 2021. Diagnosing performance issues in microservices with heterogeneous data source. In *2021 IEEE ISPA/BDCloud/SocialCom/SustainCom*.
- [8] Azam Ikram et al. 2022. Root Cause Analysis of Failures in Microservices through Causal Discovery. In *NeurIPS'22*.
- [9] Cheryl Lee et al. 2023. Eadro: An End-to-End Troubleshooting Framework for Microservices on Multi-source Data. In *ICSE'23*.
- [10] Mingjie Li et al. 2022. Causal Inference-Based Root Cause Analysis for Online Service Systems with Intervention Recognition. In *KDD'22*.
- [11] Zeyan Li et al. 2021. Practical Root Cause Localization for Microservice Systems via Trace Analysis. In *IWQOS'21*.
- [12] Zeyan Li et al. 2022. Constructing large-scale real-world benchmark datasets for aiops. *arXiv preprint arXiv:2208.03938* (2022).
- [13] Cheng-Ming Lin and et al. 2024. Root Cause Analysis in Microservice Using Neural Granger Causal Discovery. In *AAAI'24*.
- [14] Chenghao Liu et al. 2023. PyRCA: A Library for Metric-based Root Cause Analysis. *arXiv preprint arXiv:2306.11417* (2023).
- [15] Luan Pham, Huong Ha, and Hongyu Zhang. 2024. Root Cause Analysis for Microservices based on Causal Inference: How Far Are We?. In *ASE'24*. IEEE.
- [16] Luan Pham et al. 2024. BARO: Robust Root Cause Analysis for Microservices via Multivariate Bayesian Online Change Point Detection. In *FSE'24*.
- [17] Guangba Yu et al. 2021. Microrank: End-to-end latency issue localization with extended spectrum analysis in microservice environments. In *WWW'21*.
- [18] Guangba Yu et al. 2023. Nezha: Interpretable Fine-Grained Root Causes Analysis for Microservices on Multi-modal Observability Data. In *FSE'23*.
- [19] Shenglin Zhang et al. 2023. Robust failure diagnosis of microservice system through multimodal data. *TSC'23* (2023).